

♦ Travailler avec l'application

Installer. L'application demande aussi Node.js 20 ou plus, avec npm (par exemple `mise en place --global node@lts`), et SQLite 3 (`sqlite3 --version`). Elle se clone **dans** le dossier des supports : les commandes `cp` des steps en dépendent.

```
cd formation-ruby
git clone https://github.com/Tuxii/workshops.git
cd workshops
bin/setup --skip-server          # installe les gems et les paquets npm, crée la base
bin/rails runner 'puts Rails.version' # affiche 8.1.4
bin/dev                          # puis http://localhost:3000 ; Ctrl+C pour arrêter
```

Les branches. `workshops/` a une branche par step : `step-NN` contient l'application **à la fin** de l'étape NN, c'est la solution ; `step-00` est le point de départ. Vous travaillez sur votre propre branche : `git switch -c mon-travail origin/step-00`.

Les consignes sont dans `rails/steps/`. La section « Step NN » de ce support (puis de `3-active-record.pdf`, `4-requetes-et-hotwire.pdf` et `5-tester-et-reprendre.pdf`) accompagne le step NN : ouvrez-la quand la consigne demande une syntaxe, relisez-la en fin de step.

Les fichiers fournis. Depuis `workshops/`, lancez la commande `cp` donnée par l'énoncé, au moment où il la donne, par exemple `cp -r ../rails/fournis/step-NN/. .`. La copie écrase les fichiers de même nom : committez avant, `git diff` montrera ensuite ce qui a changé.

Comparer avec la solution. Les solutions sont publiées au fil de la formation :

```
git fetch                                # récupère les solutions publiées
git diff origin/step-NN                 # compare votre code avec la solution
git diff origin/step-NN -- app/models   # seulement les modèles
git show origin/step-NN:app/models/session.rb # lit un fichier de la solution, sans changer de branche
```

Repartir d'une solution. Mettez d'abord votre travail de côté, sinon `git switch` refusera de changer de branche :

```
git add -A && git commit -m "Travail en cours" # ou : git stash -u
git switch -c rattrapage-NN origin/step-NN
bundle install                                # si la branche utilise d'autres gems
bin/rails db:reset                            # recrée la base depuis db/schema.rb et rejoue les
seeds                                         #
bin/dev                                       # après Ctrl+C sur le bin/dev en cours
```

Lancez `bin/rails db:reset` à chaque changement de branche, puis relancez `bin/dev` : sinon le serveur garde ouverte l'ancienne base supprimée et affiche des données périmées. N'utilisez pas `bin/setup` ni `bin/rails db:prepare` à la place : ils lanceraient les migrations de la branche sur votre base existante et s'arrêteraient sur `table "workshops" already exists`.

Pour retrouver votre travail : `git switch mon-travail`, `git stash pop` si vous avez utilisé `git stash -u`, puis `bin/rails db:reset` et `bin/dev` relancé.

Les exemples décrivent une bibliothèque : des livres (`Book`), leurs auteurs (`Author`), des membres (`Member`) et leurs emprunts (`Loan`).

♦ Step 01 : l'application et Active Record

Structure et conventions de nommage

<code>app/</code>	le code, rangé par rôle : <code>models/</code> , <code>controllers/</code> , <code>views/</code> , <code>helpers/...</code>
<code>config/routes.rb</code>	toutes les routes, dans un seul fichier
<code>config/</code>	la configuration (<code>environments/</code> , <code>initializers/...</code>), les traductions (<code>locales/</code>)
<code>db/</code>	<code>migrate/</code> (les migrations), <code>schema.rb</code> (l'état du schéma), <code>seeds.rb</code>
<code>test/</code>	les tests
<code>Gemfile</code>	les dépendances, gérées par Bundler ; versions figées dans <code>Gemfile.lock</code>
<code>bin/rails</code>	la ligne de commande : générateurs, migrations, console, serveur

Un nom suffit à retrouver tout le reste :

<code>Book</code>	le modèle, dans <code>app/models/book.rb</code>
<code>books</code>	sa table : au pluriel, en <code>snake_case</code>
<code>book_id</code>	une clé étrangère vers un livre
<code>BooksController</code>	son contrôleur, dans <code>app/controllers/books_controller.rb</code>
<code>app/views/books/</code>	les vues de ce contrôleur : <code>index.html.erb</code> , <code>show.html.erb...</code>
<code>resources :books</code>	ses routes, et leurs helpers : <code>books_path</code> , <code>book_path(book)...</code>

Un nom composé suit la même règle (`LoanRequest` , table `loan_requests` , fichier `loan_request.rb`), avec le pluriel de l'anglais. En cas de doute : `"Person".tableize` renvoie `"people"` .

Migrations et `db/schema.rb`

```
bin/rails generate model Book title:string
```

```
class CreateBooks < ActiveRecord::Migration[8.1]
  def change
    create_table :books do |t|
      t.string :title
      t.timestamps # created_at et updated_at, remplies par Rails
    end
  end
end
```

La clé primaire `id` est implicite. `bin/rails db:migrate` joue les migrations en attente, puis réécrit `db/schema.rb` en lisant la base : ce fichier décrit l'état courant du schéma et ne se modifie jamais à la main.

Le modèle ne déclare rien (`class Book < ApplicationRecord; end`). Active Record lit les colonnes de la table à l'exécution, et génère les méthodes d'accès (`title` , `title= ...`) dans un module inclus dans la classe. Les attributs d'un modèle se lisent donc dans `db/schema.rb` , pas dans le fichier du modèle.

Créer, retrouver, modifier, supprimer

```
book = Book.new(title: "Germinal") # aucun SQL : un objet en mémoire, id vaut nil
book.save                         # INSERT INTO "books" (...) VALUES (...) RETURNING "id"
Book.create(title: "Nana")        # new, puis save
Book.find(3)                      # SELECT "books".* FROM "books" WHERE "books"."id" = 3 LIMIT 1
                                  # lève ActiveRecord::RecordNotFound si rien n'est trouvé

Book.find_by(title: "Nana")       # ... WHERE "books"."title" = 'Nana' LIMIT 1, ou nil
Book.where(title: "Nana")         # une relation : SQL seulement quand on la parcourt
Book.order(:title)               # ... ORDER BY "books"."title" ASC
book.update(title: "Nana (poche)") # UPDATE "books" SET "title" = ..., "updated_at" = ... WHERE ...
book.destroy                     # DELETE FROM "books" WHERE "books"."id" = 3 ; l'objet reste, gelé
book.reload                      # relit la ligne en base ; RecordNotFound si elle a disparu
```

En console, `Book.where(...)` affiche pourtant un `SELECT ... /* loading for pp */ LIMIT 11` : c'est l'affichage du résultat qui parcourt la relation. `relation = Book.where(title: "Nana"); nil` n'exécute rien.

save écrit tout de suite

`save` écrit en base immédiatement :

```
TRANSACTION (0.0ms) BEGIN immediate TRANSACTION
Book Create (0.1ms) INSERT INTO "books" ("title", "created_at", "updated_at") VALUES (...) RETURNING
"id"
TRANSACTION (0.0ms) COMMIT TRANSACTION
```

L'objet connaît ses modifications, et `save` n'écrit que les colonnes modifiées : après `book.title = "Germinal (poche)"`, `book.changed?` renvoie `true` et `book.changes` renvoie `{"title" => ["Germinal", "Germinal (poche)"]}`.

La console et le SQL dans les logs

`bin/rails console` charge l'application ; `bin/rails console --sandbox` annule tout à la sortie. En développement, la console affiche chaque requête SQL sous l'appel qui l'a déclenchée, `reload!` recharge le code modifié, et `relation.to_sql` montre le SQL d'une relation sans l'exécuter. Le serveur et les tests écrivent les mêmes lignes dans `log/development.log` et `log/test.log`.

♦ Step 02 : migrations, validations, chargement des classes

Faire évoluer une table

Chaque changement de schéma est une nouvelle migration. Le nom `Add...ToBooks` indique la table au générateur :

```
bin/rails generate migration AddDetailsToBooks pages:integer copies:integer
```

```
class AddDetailsToBooks < ActiveRecord::Migration[8.1]
  def change
    add_column :books, :pages, :integer
    add_column :books, :copies, :integer, default: 1, null: false # options ajoutées à la main
  end
end
```

Rails sait inverser `add_column`, `create_table` ou `add_index` : avec une méthode `change`, la migration est réversible. Une migration déjà jouée ailleurs ne se modifie plus.

```
bin/rails db:migrate      # joue les migrations en attente, réécrit db/schema.rb
bin/rails db:migrate:status # up ou down, pour chaque migration
bin/rails db:rollback     # défait la dernière
bin/rails db:reset        # recrée la base depuis db/schema.rb, puis joue les seeds
```

Validations et errors

```
class Book < ApplicationRecord
  validates :title, presence: true, length: { maximum: 200 }
  validates :pages, numericality: { only_integer: true, greater_than: 0 }, allow_nil: true
end

book = Book.new(title: "")
book.valid?           # => false : exécute les validations et remplit errors
book.errors.full_messages # => ["Title doit être rempli(e)"]
book.errors[:title]    # => ["doit être rempli(e)"]
book.save             # => false, rien n'est écrit
book.save!            # lève ActiveRecord::RecordInvalid
```

Même convention partout : sans `!`, la méthode renvoie `false` ; avec `!` (`save!`, `create!`, `update!`), elle lève une exception. `create` renvoie l'objet même s'il n'a pas été enregistré : `persisted?` le dit. Les messages passent par `I18n`, dans la langue par défaut de l'application.

Pour chaque colonne, Active Record génère aussi un prédicat (`title?`, vrai si la valeur est présente) et des méthodes de suivi (`title_was`, `title_changed?`). Les méthodes que vous écrivez s'y ajoutent ; `book.method(:title?).owner` dit qui a défini une méthode.

Un nom, un fichier

Rails charge les classes de `app/` sans `require` : le chemin du fichier donne le nom de la classe.

<code>app/models/book.rb</code>	<code>Book</code>
<code>app/models/loan_request.rb</code>	<code>LoanRequest</code>
<code>app/services/loans/renewal.rb</code>	<code>Loans::Renewal</code>

Pour trouver une classe dans une application, convertissez son nom en chemin (`"Loans::Renewal".underscore`). En développement, le code modifié est rechargé sans redémarrer : `reload!` en console, automatiquement pour le serveur.

Seeds

`bin/rails db:seed` exécute `db/seeds.rb`. Écrit avec `create!`, il crée des doublons à chaque exécution. `find_or_create_by!` cherche d'abord (`SELECT`), et ne crée (`INSERT`) que si rien n'est trouvé :

```
Book.find_or_create_by!(title: "Les Misérables") do |book|
  book.assign_attributes(pages: 1900, copies: 2) # le bloc ne s'exécute qu'à la création
end
```

♦ Step 03 : routes, contrôleur, layout

Le cycle d'une requête

<code>GET /books/3</code>	
→ <code>config/routes.rb</code>	<code>resources :books : BooksController#show</code> , avec <code>params[:id] = "3"</code>
→ <code>BooksController</code>	les <code>before_action</code> , puis la méthode <code>show : @book = Book.find(params[:id])</code>
→ la vue	pas de <code>render</code> : <code>app/views/books/show.html.erb</code>
→ le layout	la vue est insérée dans <code>app/views/layouts/application.html.erb</code>
→ la réponse	<code>200</code> , une page HTML complète

Le routeur associe un verbe et un chemin à un couple `contrôleur#action`. Rails crée une instance du contrôleur pour chaque requête et appelle la méthode publique qui porte le nom de l'action. `bin/rails generate controller Pages home` écrit le contrôleur, l'action, sa vue et une route.

Dans `config/routes.rb`, `root "pages#home"` envoie `GET /` vers `PagesController#home`, et `get "about", to: "pages#about"` déclare une route isolée. `bin/rails routes` liste toutes les routes, y compris celles que Rails et les gems ajoutent ; `-g` filtre sur un mot, `-c` sur un contrôleur.

Ce que Rails fait sans qu'on lui demande

```
class PagesController < ApplicationController
  def home
    @books_count = Book.count
  end
end
```

- **Rendu implicite** : une action qui n'appelle ni `render` ni `redirect_to` rend la vue qui porte son nom, ici `app/views/pages/home.html.erb`.
- **Variables d'instance** : avant le rendu, Rails copie les variables d'instance du contrôleur (`@books_count`) dans la vue. C'est la seule interface entre les deux, sans déclaration : une variable mal orthographiée vaut `nil` dans la vue, sans erreur.

Vues, layout, partials

En ERB, `<%= ... %>` affiche le résultat en échappant le HTML, `<% ... %>` exécute sans afficher (un `if`, une boucle), et seul `<%= ... %>` neutralise du Ruby : dans un commentaire HTML, `<!-- <%= ... %> -->`, le code s'exécute quand même.

Le layout `app/views/layouts/application.html.erb` enveloppe toutes les pages : la vue de l'action est insérée à la place de `<%= yield %>`. Un partial est un fragment de vue dont le nom de fichier commence par `_`, appelé sans le `_` : `<%= render "layouts/navbar" %>` rend `app/views/layouts/_navbar.html.erb`. Les helpers de Rails (`link_to`, `pluralize` ...) sont disponibles dans toutes les vues.

Lire le log d'une requête

```
Started GET "/" for 127.0.0.1 at ...
Processing by PagesController#home as HTML
  Book Count (0.1ms)  SELECT COUNT(*) FROM "books"
  ↳ app/controllers/pages_controller.rb:3:in 'PagesController#home'
  Rendered pages/home.html.erb within layouts/application (Duration: 0.4ms | GC: 0.0ms)
  Rendered layout layouts/application.html.erb (Duration: 16.7ms | GC: 3.0ms)
Completed 200 OK in 56ms (Views: 18.0ms | ActiveRecord: 2.2ms (1 query, 0 cached) | GC: 22.4ms)
```

Dans l'ordre : la requête, le contrôleur et l'action, chaque requête SQL, les templates rendus, puis le statut et le bilan, dont le nombre de requêtes SQL. En développement, la ligne `↳` donne la ligne de code qui a lancé la requête SQL au-dessus d'elle. Chaque générateur écrit aussi un test, que `bin/rails test` lance avec les autres.

♦ Step 04 : CRUD et scaffold

resources et les 7 actions

```
resources :books

GET    /books          index    books_path
GET    /books/new      new      new_book_path
POST   /books          create   books_path
GET    /books/:id      show     book_path(book)
GET    /books/:id/edit edit     edit_book_path(book)
PATCH /books/:id      update   book_path(book)      (et PUT)
DELETE /books/:id      destroy  book_path(book)
```

Sept actions, huit routes. Chaque helper `_path` a une version `_url`, avec le domaine. `link_to "Voir", book` et `redirect_to book` déduisent `book_path(book)` de la classe de l'objet.

Le code généré par le scaffold

`bin/rails generate scaffold Book title pages:integer` écrit la migration, le modèle, `resources :books`, le contrôleur, six vues (`index`, `show`, `new`, `edit`, `_form`, `_book`), un helper, des tests et des fixtures, et une API JSON (vues `.json.jbuilder`). Sur un modèle existant, `--skip` saute les fichiers qui existent déjà. Extrait du contrôleur :

```
class BooksController < ApplicationController
  before_action :set_book, only: [:index, :show, :edit, :update, :destroy]

  def create
    @book = Book.new(book_params)

    respond_to do |format|
      if @book.save
        format.html { redirect_to @book, notice: "Book was successfully created." }
        format.json { render :show, status: :created, location: @book }
      else
        format.html { render :new, status: :unprocessable_content }
        format.json { render json: @book.errors, status: :unprocessable_content }
      end
    end
  end
end
```

Après les autres actions (`index`, `show`, `new`, `edit`, `update`, `destroy`), les méthodes utilitaires :

```
private
def set_book
  @book = Book.find(params.expect(:id))
end

def book_params
  params.expect(book: [:title, :pages])
end
end
```

Un contrôleur se lit toujours dans cet ordre : les `before_action`, les actions publiques, puis sous `private` les méthodes utilitaires.

- `before_action :set_book, only: [...]` s'exécute avant les actions listées. `find` lève `RecordNotFound` si l'id n'existe pas : une 404 en production.
- `respond_to` choisit la réponse selon le format demandé : HTML pour le navigateur, JSON pour l'API générée.
- Succès : `redirect_to`. Le navigateur fait une nouvelle requête `GET`, et le message `notice` l'accompagne.
- Échec : `render :new` avec le statut 422 (`:unprocessable_content`). Le formulaire est rendu dans la même requête, avec l'objet, ses erreurs et la saisie. Le step 14 revient sur ce statut, et sur le `:see_other` de `update` et `destroy`.
- Dans du code plus ancien : `params.require(:book).permit(...)` au lieu de `expect`, `:unprocessable_entity` au lieu de `:unprocessable_content`.

Strong parameters

`params` contient tout ce que le client a envoyé, champs forgés compris. `params.expect(book: [:title, :pages])` exige une clé `book` contenant un hash, et n'en garde que les clés listées : les autres sont écartées sans erreur. Sans clé `book`, la réponse est une 400. Passer à `Book.new` des paramètres non filtrés lève `ActiveModel::ForbiddenAttributesError`.

Formulaires, partials, helpers, flash

`form_with model: @book` construit un formulaire `POST /books` pour un objet neuf, `PATCH /books/3` pour un objet enregistré (un champ caché `_method` porte le verbe). Ses champs s'appellent `book[title]`, `book[pages]` : c'est le hash que `params.expect` filtre.

`render @books` rend `app/views/books/_book.html.erb` une fois par élément, avec une variable locale `book`. Les autres variables locales se passent en hash : `render "shared/errors", record: @book` donne au partial une variable `record`,

dont il affiche les erreurs :

```
<% record.errors.full_messages.each do |message| %>
  <li><%= message %></li>
<% end %>
```

Chaque méthode d'un module de `app/helpers/` est disponible dans toutes les vues, sans déclaration. Pour produire du HTML dans un helper, `tag.span("Disponible", class: "badge")` échappe le contenu. `<%= %>` échappe les chaînes ; un `html_safe` sur une chaîne construite avec des données saisies ouvre une faille XSS.

`redirect_to @book, notice: "Livre créé."` range le message dans le flash (`flash[:notice]`), qui survit à la redirection et disparaît après la requête suivante. Le layout l'affiche. `alert:` fonctionne de la même façon.

♦ Pour aller plus loin : la documentation officielle

Step 01 - L'application et la découverte d'Active Record

- [Getting Started with Rails](https://guides.rubyonrails.org/getting_started.html)
guides.rubyonrails.org/getting_started.html
- [Active Record Migrations](https://guides.rubyonrails.org/active_record_migrations.html)
guides.rubyonrails.org/active_record_migrations.html
- [Active Record Basics](https://guides.rubyonrails.org/active_record_basics.html)
guides.rubyonrails.org/active_record_basics.html
- [Active Record Query Interface](https://guides.rubyonrails.org/active_record_querying.html)
guides.rubyonrails.org/active_record_querying.html
- [Active Record Basics - Active Record Pattern](https://guides.rubyonrails.org/active_record_basics.html#the-active-record-pattern)
guides.rubyonrails.org/active_record_basics.html#the-active-record-pattern
- [Dirty tracking](https://api.rubyonrails.org/classes/ActiveModel/Dirty.html)
api.rubyonrails.org/classes/ActiveModel/Dirty.html

Step 02 - Enrichir Workshop, comprendre où sont branchées les choses

- [Active Record Migrations](https://guides.rubyonrails.org/active_record_migrations.html#creating-a-standalone-migration)
guides.rubyonrails.org/active_record_migrations.html#creating-a-standalone-migration
- [Active Record Validations](https://guides.rubyonrails.org/active_record_validations.html)
guides.rubyonrails.org/active_record_validations.html
- [I18n - Active Record Models](https://guides.rubyonrails.org/i18n.html#active-record-models)
guides.rubyonrails.org/i18n.html#active-record-models
- [ActiveRecord::AttributeMethods::Query](https://api.rubyonrails.org/classes/ActiveRecord/AttributeMethods/Query.html)
api.rubyonrails.org/classes/ActiveRecord/AttributeMethods/Query.html
- [Autoloading and Reloading Constants](https://guides.rubyonrails.org/autoloading_and_reloading_constants.html)
guides.rubyonrails.org/autoloading_and_reloading_constants.html
- [Seeds](https://guides.rubyonrails.org/active_record_migrations.html#migrations-and-seed-data)
guides.rubyonrails.org/active_record_migrations.html#migrations-and-seed-data
- [find_or_create_by](https://api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_create_by)
api.rubyonrails.org/classes/ActiveRecord/Relation.html#method-i-find_or_create_by

Step 03 - Routes, contrôleur, layout

- [Rails Routing from the Outside In](https://guides.rubyonrails.org/routing.html)
guides.rubyonrails.org/routing.html
- [Layouts and Rendering](https://guides.rubyonrails.org/layouts_and_rendering.html#rendering-by-default-convention-over-configuration-in-action)
guides.rubyonrails.org/layouts_and_rendering.html#rendering-by-default-convention-over-configuration-in-action
- [Action Controller Overview](https://guides.rubyonrails.org/action_controller_overview.html)
guides.rubyonrails.org/action_controller_overview.html
- [Structuring Layouts](https://guides.rubyonrails.org/layouts_and_rendering.html#structuring-layouts)
guides.rubyonrails.org/layouts_and_rendering.html#structuring-layouts
- [Bootstrap Navbar](https://getbootstrap.com/docs/5.3/components/navbar/)
getbootstrap.com/docs/5.3/components/navbar/
- [Functional Testing for Controllers](https://guides.rubyonrails.org/testing.html#functional-testing-for-controllers)
guides.rubyonrails.org/testing.html#functional-testing-for-controllers

Step 04 - CRUD des ateliers, et lire du code généré

- [Generating Code](#)

guides.rubyonrails.org/command_line.html#generating-code

- **Strong Parameters**

guides.rubyonrails.org/action_controller_overview.html#strong-parameters

- **Resource Routing**

guides.rubyonrails.org/routing.html#resource-routing-the-rails-default

- **Action View Helpers**

guides.rubyonrails.org/action_view_helpers.html

- **Bootstrap Badges**

getbootstrap.com/docs/5.3/components/badge/

- **Using Partial**

guides.rubyonrails.org/layouts_and_rendering.html#using-partials

- **The Flash**

guides.rubyonrails.org/action_controller_overview.html#the-flash

- **Displaying Validation Errors in Views**

guides.rubyonrails.org/active_record_validations.html#displaying-validation-errors-in-views

- **Ordering Records**

guides.rubyonrails.org/active_record_querying.html#ordering-records